BLOQUE III - TEMA 3

DESARROLLO DE SISTEMAS

Lenguajes de interrogación de bases de datos. Estándar ANSI SQL. Procedimientos almacenados. Eventos y disparadores.

Supera. by thePower

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2024.

ÍNDICE

1. LENGUAJES DE BASES DE DATOS. ESTÁNDAR ANSI/SQL 5
1.1. Historia y evolución del estándar 5
1.2. Estándar ANSI SQL 6
2. SINTAXIS BÁSICA DE SQL 8
2.1. SELECT 8
2.2. Filtrado de resultados: WHERE, BETWEEN, IN, LIKE 9
2.3. Ordenación y agrupamiento: ORDER BY, GROUP BY, HAVING 10
2.4. Funciones de agregación: COUNT, SUM, AVG, MAX, MIN 12
2.5. JOINs: INNER, LEFT, RIGHT, FULL 13
2.6. Combinación de resultados: UNION, INTERSECT, EXCEPT 15
2.7. Tipos de datos en SQL 17
2.8. Operadores y funciones integradas 19
2.9. Subconsultas y consultas anidadas 20
2.10. Vistas, índices y restricciones 23
3. LENGUAJES DE BASES DE DATOS 24
3.1. Lenguaje de definición de datos (DDL) 24
3.2. Lenguaje de manipulación de datos (DML) 28
3.3 Lenguaje de Control de Datos (DCL) 30
3.4. OQL: Object Query Language 31
4. PROCEDIMIENTOS ALMACENADOS 32
4.1. Sintaxis básica (CREATE PROCEDURE) 32
4.2. Parámetros de entrada y salida 33
4.3. Variables locales, condicionales y bucles 35
4.3.1. Variables locales 35
4.3.2. Estructuras condicionales 36
4.3.3. Bucles 37

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

5. EVENTOS Y DISPARADORES (TRIGGERS)..... 39

5.1. Definición de eventos y triggers 39

5.2. Sintaxis de un Trigger en SQL 39

5.3. Disparadores BEFORE y AFTER 40

5.4. Pseudoregistros NEW y OLD 40

5.5. Tipos de eventos: INSERT, UPDATE, DELETE 41

6. CONTROL DE TRANSACCIONES Y NIVELES DE AISLAMIENTO 42

6.1. Instrucciones de control de transacciones..... 42

6.2. Problemas de concurrencia..... 42

6.3. Niveles de aislamiento de transacciones (ANSI SQL) 43

6.4. Control de transacciones: sentencias clave en SQL 43

Significado de los iconos que aparecen dentro de los TEMAS:



Examen


Importante


Recordatorio


Atención

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

1. LENGUAJES DE BASES DE DATOS. ESTÁNDAR ANSI/SQL

Los **lenguajes de bases de datos** son subconjuntos del lenguaje SQL que permiten interactuar con los sistemas de gestión de bases de datos (SGBD). Se dividen en varias categorías, cada una con un propósito específico: **definir estructuras**, **manipular datos**, **controlar el acceso** o **consultar información**. Comprender estos lenguajes es clave para gestionar eficazmente cualquier base de datos relacional.

Estos lenguajes constituyen una parte esencial de los **lenguajes de manipulación de datos (DML)** y se fundamentan en los **principios teóricos del modelo relacional**, como el álgebra relacional y el cálculo relacional.

Características generales

- Permiten **extraer información** sin modificar el contenido de la base de datos.
- Suelen estar **basados en SQL**, aunque existen otros enfoques teóricos como el **cálculo relacional** y el **álgebra relacional**.

El lenguaje **SQL (Structured Query Language)** es el lenguaje estándar para la gestión y consulta de bases de datos relacionales. Su uso está regulado por el estándar **ANSI/ISO**, lo que garantiza compatibilidad entre distintos sistemas gestores (SGBD). SQL permite definir estructuras, manipular datos, controlar accesos y realizar consultas. A continuación se desarrolla su historia, sintaxis y componentes clave según el estándar ANSI.

SQL (Structured Query Language) es el **lenguaje estándar** utilizado para gestionar y consultar bases de datos relacionales. Fue desarrollado inicialmente por IBM en los años 70 y, posteriormente, **normalizado por el ANSI (American National Standards Institute)** y el ISO (Organización Internacional de Normalización).

El término **ANSI SQL** hace referencia a las **versiones normalizadas** del lenguaje SQL publicadas por estas organizaciones, con el objetivo de establecer una **base común y compatible** entre distintos sistemas gestores de bases de datos (SGBD), como Oracle, SQL Server, MySQL o PostgreSQL.

Objetivos del estándar

- Definir un **lenguaje común y portable** para interactuar con bases de datos relacionales.
- Establecer la **sintaxis y semántica de las instrucciones** SQL.
- Facilitar la **interoperabilidad** entre distintos SGBD.
- Permitir **consultas, manipulación, definición y control de acceso** a los datos de forma estructurada y declarativa.

1.1. HISTORIA Y EVOLUCIÓN DEL ESTÁNDAR

SQL fue desarrollado inicialmente por IBM en los años 70 como parte del proyecto **System R**, con el nombre **SEQUEL**. Posteriormente, fue estandarizado por **ANSI** en 1986 y por **ISO** en 1987.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Desde entonces, ha evolucionado mediante versiones sucesivas que han añadido funcionalidades progresivamente.

El lenguaje SQL ha sido ampliado y revisado en varias versiones del estándar ANSI/ISO. Algunas de las más relevantes son:

Versión	Año	Nombre/Características
SQL-86	1986	Primera versión estándar ANSI.
SQL-89	1989	Añade mejoras menores (referencias cruzadas, precisión numérica).
SQL-92	1992	Muy extendido; incluye JOIN, CHECK, IN, UNION, subconsultas.
SQL:1999	1999	Añade soporte a programación estructurada (procedures, triggers).
SQL:2003	2003	Introduce XML, tipos autoincrementales (IDENTITY).
SQL:2011	2011	Añade soporte a consultas temporales (PERIOD, SYSTEM_TIME).
SQL:2016	2016	Incluye mejoras para JSON, funciones analíticas y agregadas.

1.2. ESTÁNDAR ANSI SQL

Partes principales del lenguaje ANSI SQL:

ANSI SQL se divide en varios sublenguajes funcionales:



Subconjunto	Función principal	Comandos
DDL (Data Definition Language)	Definir estructuras de datos	CREATE, ALTER, DROP
DML (Data Manipulation Language)	Manipular datos existentes	INSERT, UPDATE, DELETE
DCL (Data Control Language)	Gestionar permisos y seguridad	GRANT, REVOKE
QL / QBL (Query Language)	Consultar y recuperar información	SELECT, JOIN, GROUP BY
TCL (Transaction Control Language)	Controlar transacciones (consistencia, rollback, commit)	BEGIN, COMMIT, ROLLBACK

Aunque el estándar ANSI SQL proporciona una base común para el manejo de bases de datos relacionales, **cada sistema gestor (SGBD) incorpora extensiones propias** que amplían su funcionalidad y adaptan el lenguaje a necesidades específicas.

Estas extensiones afectan tanto a la sintaxis como a las capacidades del sistema, permitiendo el uso de **tipos de datos personalizados, funciones especiales, mecanismos de control de flujo y herramientas avanzadas de programación**.

A continuación, se describen brevemente las principales extensiones de algunos de los SGBD más utilizados: Oracle, MySQL, SQL Server y PostgreSQL. Estas diferencias deben tenerse en cuenta especialmente cuando se trabaja en entornos con múltiples tecnologías o se requiere portabilidad entre plataformas.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Oracle:**
  - **Extiende el estándar ANSI SQL** mediante **su lenguaje PL/SQL** (Procedural Language/SQL), que permite definir procedimientos, funciones, paquetes, cursores y estructuras de control de flujo como IF, LOOP o EXCEPTION.
  - Utiliza **tipos propio**s como NUMBER, VARCHAR2 y CLOB, e incorpora funciones avanzadas como SYSDATE, NVL, y DECODE. Para autoincrementar valores, Oracle emplea secuencias (SEQUENCE) y, desde versiones recientes, admite también IDENTITY.
  - La limitación de filas se realiza tradicionalmente con **ROWNUM** y en versiones modernas con **FETCH FIRST.**
- **MySQL:**
  - **MySQL amplía ANSI SQL** con características sencillas y de fácil uso como el tipo de dato AUTO_INCREMENT para claves primarias, funciones de cadena como CONCAT() y operadores de fecha como NOW().
  - Permite el uso de procedimientos y disparadores (TRIGGER) mediante una sintaxis más simple, y utiliza su propio sistema de control de errores dentro de bloques BEGIN ... END.
  - Para limitar resultados, utiliza LIMIT, muy común en aplicaciones web.
- **SQL Server:**
  - **SQL Server, de Microsoft**, utiliza el lenguaje **Transact-SQL (T-SQL)** como extensión de SQL estándar. T-SQL permite estructuras avanzadas como variables locales, control de flujo (IF, WHILE, TRY...CATCH), procedimientos almacenados, funciones definidas por el usuario y cursores.
  - En lugar de LIMIT, utiliza la cláusula TOP para limitar filas.
- **PostgreSQL:**
  - **PostgreSQL amplía SQL mediante el lenguaje PL/pgSQL,** que combina la potencia de un lenguaje procedural con el estándar relacional.
  - Utiliza tipos como SERIAL y BIGSERIAL para autoincremento, y es especialmente destacable por su adherencia al estándar ANSI SQL, manteniendo una gran compatibilidad.
  - Ofrece funciones avanzadas como COALESCE, ILIKE (búsqueda insensible a mayúsculas), soporte a JSON, arrays, y funciones agregadas definidas por el usuario.
  - Para limitar filas, emplea LIMIT y OFFSET, y permite trabajar con estructuras como RETURNING, muy útil en inserciones o actualizaciones complejas.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2. SINTAXIS BÁSICA DE SQL

La sintaxis SQL es de tipo **declarativo**, lo que significa que el usuario indica **qué** desea obtener, no **cómo** se debe procesar. Toda sentencia SQL se compone de palabras clave, identificadores (como nombres de tablas o columnas) y condiciones.

2.1. SELECT

La instrucción SELECT es el **comando más utilizado en SQL**, ya que permite **recuperar información** de una o varias tablas. A través de sus distintas cláusulas, el usuario puede **filtrar, ordenar, agrupar, unir** y **combinar** datos de forma flexible y eficiente. En este apartado se desglosan sus componentes más importantes.



Estructura básica de una consulta:

```
SELECT columnas
FROM tabla
[WHERE condición]
[GROUP BY columna]
[HAVING condición]
[ORDER BY columna];
```

Cada cláusula tiene un propósito específico:

- **SELECT: indica las columnas que se desean visualizar.**
- **FROM: especifica la tabla o tablas de origen.**
- WHERE: filtra filas según condiciones.
- GROUP BY: agrupa resultados para funciones de agregación.
- HAVING: filtra los grupos formados.
- ORDER BY: ordena el resultado final.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2.2. FILTRADO DE RESULTADOS: WHERE, BETWEEN, IN, LIKE

La cláusula **WHERE** permite **filtrar filas** en una consulta SQL, devolviendo únicamente aquellas que **cumplen una o varias condiciones lógicas**.

- **"=" Igualdad:** Filtra filas que **coinciden exactamente** con un valor.

```
SELECT * FROM nombreTabla
WHERE nombreColumna = valor;
```

- **BETWEEN ... AND ... :** Selecciona valores que estén dentro de un rango (inclusive en ambos extremos).

```
SELECT * FROM nombreTabla
WHERE nombreColumna BETWEEN valor1 AND valor2;
```

- **IN (...):** Permite comprobar si un valor pertenece a una lista específica de opciones.

```
SELECT * FROM nombreTabla
WHERE nombreColumna IN (valor1, valor2, valor3);
```

- **LIKE — Búsqueda por patrones: Busca coincidencias en cadenas de texto usando comodines:**
  - % representa **cualquier número de caracteres**.
  - _ representa **un único carácter**.

```
SELECT * FROM nombreTabla
WHERE nombreColumna LIKE 'loquesea%';
```

Ejemplo:

```
WHERE CodigoPostal LIKE '28_00'
```

Filtra códigos postales como 28000, 28100, etc.

- **IS NULL:** Se utiliza para filtrar registros cuyo valor en una columna es `NULL`.

```
SELECT * FROM nombreTabla
WHERE nombreColumna IS NULL;
```

- **AND, OR, NOT — Lógica booleana:** Permiten combinar condiciones múltiples:
  - AND: ambas condiciones deben cumplirse.
  - OR: basta con que se cumpla una.
  - NOT: niega una condición.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Ejemplo con AND:

```
SELECT * FROM Empleados
WHERE DepartamentoID = 10 AND Salario > 30000;
```

Ejemplo con OR:

```
SELECT * FROM Empleados
WHERE DepartamentoID = 10 OR DepartamentoID = 20;
```

Ejemplo con NOT:

```
SELECT * FROM Empleados
WHERE NOT Salario BETWEEN 20000 AND 30000;
```

2.3. ORDENACIÓN Y AGRUPAMIENTO: ORDER BY, GROUP BY, HAVING

Estas tres cláusulas permiten **organizar y resumir los resultados** de una consulta SQL. Mientras ORDER BY se encarga de la **presentación del resultado**, GROUP BY permite **agrupar registros similares**, y HAVING actúa como un filtro **sobre los grupos formados**.

- **ORDER BY**

La cláusula ORDER BY se utiliza para **ordenar las filas devueltas por una consulta**, según una o varias columnas. El orden puede ser:

- Ascendente (ASC, por defecto)
- Descendente (DESC)

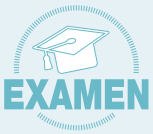
```
SELECT columnas
FROM tabla
ORDER BY columna1 [ASC|DESC], columna2 [ASC|DESC];
```

Ejemplo: Este ejemplo devuelve todos los empleados ordenados alfabéticamente por apellidos y, dentro de cada apellido, del más reciente al más antiguo según su fecha de ingreso.

```
SELECT * FROM Empleados
ORDER BY Apellidos ASC, FechaIngreso DESC;
```

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **GROUP BY**



La cláusula GROUP BY permite **agrupar filas que tienen valores idénticos** en una o varias columnas, de modo que se pueda aplicar sobre ellas **funciones de agregación** (SUM, AVG, COUNT, etc.).

```
SELECT columna_agrupada, función_agregada
FROM tabla
GROUP BY columna_agrupada;
```

Ejemplo: Este ejemplo agrupa a los empleados por departamento y devuelve cuántos hay en cada uno.

```
SELECT DepartamentoID, COUNT(*) AS NumEmpleados
FROM Empleados
GROUP BY DepartamentoID;
```

- **HAVING**

La cláusula HAVING se utiliza para **filtrar los resultados después del GROUP BY**, es decir, **sobre los grupos** ya formados. Se usa con condiciones que involucran **funciones de agregación**, lo cual no es posible con WHERE.

```
SELECT columna_agrupada, función_agregada
FROM tabla
GROUP BY columna_agrupada
HAVING condición_sobre_agregación;
```

Ejemplo: Devuelve únicamente los departamentos que tienen más de 10 empleados.

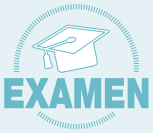
```
SELECT DepartamentoID, COUNT(*) AS Num
FROM Empleados
GROUP BY DepartamentoID
HAVING COUNT(*) > 10;
```


© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2.4. FUNCIONES DE AGREGACIÓN: COUNT, SUM, AVG, MAX, MIN

Las **funciones de agregación** permiten **resumir conjuntos de datos**, devolviendo un solo valor a partir de varias filas. Se utilizan habitualmente con GROUP BY, pero también pueden emplearse sin agrupar.

- **COUNT()**



Devuelve el **número total de filas** (o valores no nulos) de una columna.

- **COUNT(*):** cuenta **todas** las filas, incluidas las que contienen valores nulos.
- **COUNT(columna)**: cuenta solo las filas **donde la columna no sea NULL**.

```
SELECT COUNT(*) FROM tabla;

SELECT COUNT(columna) FROM tabla;
```

Ejemplo: Devuelve el número total de empleados.

```
SELECT COUNT(*) FROM Empleados;
```

- **SUM()**

Devuelve la **suma de los valores numéricos** de una columna.

```
SELECT SUM(columna) FROM tabla;
```

Ejemplo: Devuelve la suma de los salarios del departamento 10.

```
SELECT SUM(Salario) FROM Empleados

WHERE DepartamentoID = 10;
```

- **AVG()**

Calcula la **media aritmética** de los valores de una columna numérica.

```
SELECT AVG(columna) FROM tabla;
```

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Ejemplo: Devuelve el salario medio en el departamento 20.

```
SELECT AVG(Salario) FROM Empleados
WHERE DepartamentoID = 20;
```

- **MAX()**

Devuelve el **valor más alto** de una columna.

```
SELECT MAX(columna) FROM tabla;
```

Ejemplo: Devuelve el salario más alto registrado.

```
SELECT MAX(Salario) FROM Empleados;
```

- **MIN()**

Devuelve el **valor más bajo** de una columna.

```
SELECT MIN(columna) FROM tabla;
```

Ejemplo: Devuelve la fecha del primer ingreso registrado.

```
SELECT MIN(FechaIngreso) FROM Empleados;
```

- **Uso combinado**

Puedes utilizar varias funciones de agregación en una misma consulta:

```
SELECT COUNT(*), AVG(Salario), MAX(Salario), MIN(Salario)
FROM Empleados
WHERE Activo = TRUE;
```

2.5. JOINS: INNER, LEFT, RIGHT, FULL

Las **cláusulas JOIN** permiten **combinar datos de varias tablas** en una sola consulta, a partir de una **relación lógica** entre ellas (normalmente una **clave primaria** en una tabla y una **clave foránea** en otra).

Estas uniones se utilizan para **reunir información dispersa** en distintas tablas y formar una vista coherente del conjunto de datos.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **INNER JOIN – Coincidencias exactas**

Une las filas de dos tablas **solo cuando existe coincidencia** en la condición especificada (ON). Si no hay coincidencia, la fila **no aparece en el resultado**.

```
SELECT columnas

FROM tabla1

INNER JOIN tabla2

ON tabla1.columna = tabla2.columna;
```

Ejemplo: Muestra solo empleados que tienen asignado un departamento existente.

```
SELECT E.Nombre, D.Nombre AS Departamento

FROM Empleados E

INNER JOIN Departamentos D ON E.DepartamentoID = D.ID;
```

- **LEFT JOIN (o LEFT OUTER JOIN) – Todo de la izquierda**

Devuelve **todas las filas de la tabla izquierda** y solo las coincidencias de la derecha. Si no hay coincidencia, las columnas de la derecha se rellenan con NULL.

```
SELECT columnas

FROM tabla1

LEFT JOIN tabla2

ON tabla1.columna = tabla2.columna;
```

Ejemplo: Incluye todos los empleados, incluso si no tienen departamento asignado (en cuyo caso el campo Departamento será NULL).

```
SELECT E.Nombre, D.Nombre AS Departamento

FROM Empleados E

LEFT JOIN Departamentos D ON E.DepartamentoID = D.ID;
```

- **RIGHT JOIN (o RIGHT OUTER JOIN) – Todo de la derecha**

Es el opuesto del LEFT JOIN: devuelve **todas las filas de la tabla derecha**, junto con las coincidencias de la izquierda. Si no hay coincidencia, las columnas de la izquierda serán NULL.

```
SELECT columnas

FROM tabla1

RIGHT JOIN tabla2

ON tabla1.columna = tabla2.columna;
```

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Ejemplo: Devuelve todos los departamentos, incluso los que no tienen empleados asignados.

```
SELECT E.Nombre, D.Nombre AS Departamento

FROM Empleados E

RIGHT JOIN Departamentos D ON E.DepartamentoID = D.ID;
```

- **FULL JOIN (o FULL OUTER JOIN) – Todos los datos**

Devuelve **todas las filas de ambas tablas**, y en caso de que no haya coincidencia en alguna, se completan los datos con NULL. No todos los SGBD lo soportan directamente (por ejemplo, MySQL no lo permite sin emularlo con UNION).

```
SELECT columnas

FROM tabla1

FULL JOIN tabla2

ON tabla1.columna = tabla2.columna;
```

Ejemplo: Devuelve todos los empleados y todos los departamentos, coincidan o no.

```
SELECT E.Nombre, D.Nombre AS Departamento

FROM Empleados E

FULL JOIN Departamentos D ON E.DepartamentoID = D.ID;
```

2.6. COMBINACIÓN DE RESULTADOS: UNION, INTERSECT, EXCEPT

Las operaciones de **combinación de conjuntos** permiten **unir los resultados de dos o más consultas SQL**. Estas operaciones trabajan sobre conjuntos de datos que tienen **el mismo número de columnas y tipos de datos compatibles en orden**, y se ejecutan sobre las filas completas resultantes de cada consulta.

- **UNION**

La instrucción UNION **combina los resultados de dos consultas** y **elimina los duplicados**.

```
SELECT columna1, columna2, ...

FROM tabla1

UNION

SELECT columna1, columna2, ...

FROM tabla2;
```


© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Ejemplo: Este ejemplo muestra los nombres de todos los clientes, sin repetir aquellos que estén en ambas tablas.

```
SELECT Nombre FROM ClientesNacionales

UNION

SELECT Nombre FROM ClientesInternacionales;
```

- **UNION ALL**

Igual que UNION, pero **mantiene los duplicados**. Es más rápido porque no realiza comparación para eliminar repeticiones.

```
SELECT columna1 FROM tabla1

UNION ALL

SELECT columna1 FROM tabla2;
```

Ejemplo: Se obtienen todas las ocurrencias, incluso si un nombre aparece en ambas tablas.

```
SELECT Nombre FROM ClientesNacionales

UNION ALL

SELECT Nombre FROM ClientesInternacionales;
```

- **INTERSECT**

Devuelve solo las **filas que están presentes en ambas consultas**. No permite duplicados por definición.

Esta instrucción **no está soportada por algunos SGBD como MySQL**, aunque sí por PostgreSQL, SQL Server y Oracle.

```
SELECT columna1, columna2

FROM tabla1

INTERSECT

SELECT columna1, columna2

FROM tabla2;
```

Ejemplo: Devuelve los correos electrónicos que están en ambas listas.

```
SELECT Email FROM ClientesNewsletter

INTERSECT

SELECT Email FROM ClientesPremium;
```

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **EXCEPT (o MINUS en Oracle)**

Devuelve las **filas que aparecen en la primera consulta pero no en la segunda**. Es útil para obtener diferencias entre conjuntos.

En Oracle se usa la palabra clave MINUS en lugar de EXCEPT.

```
SELECT columna1, columna2
FROM tabla1
EXCEPT
SELECT columna1, columna2
FROM tabla2;
```

Ejemplo: Devuelve los DNI de clientes que no son morosos.

```
SELECT DNI FROM Clientes
EXCEPT
SELECT DNI FROM ClientesMorosos;
```

2.7. TIPOS DE DATOS EN SQL

Los **tipos de datos** son fundamentales en el diseño de bases de datos, ya que **determinan la naturaleza de los valores** que pueden almacenarse en cada columna de una tabla. Definir correctamente el tipo de dato mejora la **integridad, el rendimiento y la compatibilidad** de las operaciones SQL.

El estándar **ANSI SQL** establece un conjunto de tipos de datos básicos que todo SGBD debería soportar, aunque muchos añaden **extensiones propias**. A continuación se presentan los tipos más comunes, agrupados por categoría:

- **Tipos numéricos**

Se utilizan para almacenar **valores enteros o decimales**.

Tipo	Descripción	Ejemplo
INT o INTEGER	Número entero de tamaño estándar.	45, -7
SMALLINT	Número entero más pequeño.	127, -50
BIGINT	Número entero muy grande.	9876543210
DECIMAL(p,s)	Número exacto con precisión p y escala s.	DECIMAL(8,2) → 1234.56
NUMERIC(p,s)	Igual que DECIMAL, pero con validación más estricta.	
FLOAT	Número real aproximado, de precisión variable.	3.14159
REAL	Número en coma flotante de menor precisión.	
DOUBLE PRECISION	Mayor precisión en coma flotante.	

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Tipos de texto (cadenas de caracteres)**

Se utilizan para almacenar **letras, palabras, frases o códigos alfanuméricos**.

Tipo	Descripción	Ejemplo
CHAR(n)	Cadena de longitud fija. Se rellena con espacios si es menor.	CHAR(5) → 'A '
VARCHAR(n)	Cadena de longitud variable (máx. n).	VARCHAR(20) → 'Ana'
CLOB	Texto de gran tamaño (Character Large Object).	Documentos largos

- **Tipos de fecha y hora**

Permiten almacenar información temporal, útil para registros, planificación o caducidades.

Tipo	Descripción	Ejemplo
DATE	Fecha completa (año, mes, día).	2024-12-01
TIME	Hora (hora, minuto, segundo).	15:30:00
TIMESTAMP	Fecha y hora combinadas.	2025-04-14 09:15:30
INTERVAL	Diferencia de tiempo entre dos valores.	INTERVAL '5' DAY

- **Tipo lógico o booleano**

Almacena **valores lógicos**. No está soportado por todos los SGBD según el estándar SQL-92.

Tipo	Descripción	Ejemplo
BOOLEAN	Verdadero o falso	TRUE, FALSE

- **Tipos binarios**

Almacenan **datos no interpretables** como texto, como imágenes, archivos o valores codificados.

Tipo	Descripción	Ejemplo (en binario)
BINARY(n)	Cadena binaria de longitud fija	0x4F6B3A...
VARBINARY(n)	Cadena binaria de longitud variable	1234AB
BLOB	Objeto binario grande (Binary Large Object)	Imagen, PDF, etc.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2.8. OPERADORES Y FUNCIONES INTEGRADAS

El lenguaje SQL incluye una amplia gama de **operadores y funciones integradas** que permiten **realizar cálculos, transformaciones, filtrados y evaluaciones** sobre los datos. Estas herramientas son esenciales para construir consultas eficientes, expresivas y adaptadas a distintos tipos de datos (números, texto, fechas...).

- **Operadores**

Los **operadores** permiten realizar **cálculos y comparaciones** en las cláusulas SELECT, WHERE, HAVING y otras partes de las consultas.

Aritméticos: Se utilizan para **operar con valores numéricos**

Operador	Descripción	Ejemplo
+	Suma	`SELECT 5 + 3; → 8`
-	Resta	`SELECT 10 - 4; → 6`
*	Multiplicación	`SELECT Precio * 2 FROM Productos;`
/	División	`SELECT 20 / 4; → 5`

Comparación: Permiten comparar valores y devolver un resultado booleano (TRUE o FALSE)



Operador	Descripción	Ejemplo
=	Igual a	`WHERE Edad = 18`
<>	Distinto de	`WHERE Nombre <> 'Ana'`
>	Mayor que	`WHERE Precio > 100`
<	Menor que	`WHERE Edad < 30`
>=	Mayor o igual	`WHERE Stock >= 10`
<=	Menor o igual	`WHERE Salario <= 30000`

Lógicos: Se utilizan para combinar condiciones lógicas en consultas

Operador	Uso	Ejemplo
AND	Ambas condiciones deben cumplirse	`WHERE Edad > 18 AND Activo = TRUE`
OR	Al menos una debe cumplirse	`WHERE Ciudad = 'Madrid' OR Ciudad = 'Sevilla'`
NOT	Niega la condición	`WHERE NOT Activo`

Supera. by thePower © Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Funciones comunes**

Las **funciones integradas** permiten realizar operaciones sobre columnas y valores, y pueden clasificarse en varios grupos

Funciones numéricas

Función	Descripción	Ejemplo
ABS(x)	Valor absoluto	`ABS(-5) → 5`
ROUND(x, n)	Redondea a n decimales	`ROUND(3.14159, 2) → 3.14`
CEIL(x)	Redondea hacia arriba	`CEIL(2.1) → 3`
FLOOR(x)	Redondea hacia abajo	`FLOOR(2.9) → 2`
MOD(x, y)	Resto de la división	`MOD(10, 3) → 1`

Funciones de texto

Función	Descripción	Ejemplo
UPPER(texto)	Convierte a mayúsculas	`UPPER('hola') → 'HOLA'`
LOWER(texto)	Convierte a minúsculas	`LOWER('SQL') → 'sql'`
LENGTH(texto)	Longitud de la cadena	`LENGTH('hola') → 4`
CONCAT(a,b)	Concatena dos cadenas	`CONCAT('Ana', ' Pérez') → 'Ana Pérez'`
SUBSTRING(texto, inicio, longitud)	Extrae parte de una cadena	`SUBSTRING('informática', 1, 5) → 'infor'`

Funciones de fecha y hora

Función	Descripción	Ejemplo
CURRENT_DATE	Fecha actual	`SELECT CURRENT_DATE;`
CURRENT_TIMESTAMP	Fecha y hora actuales	`SELECT CURRENT_TIMESTAMP;`
DATEADD(intervalo, cantidad, fecha)	Suma días/meses/años	`DATEADD(DAY, 7, '2025-04-14') → 2025-04-21`
DATEDIFF(intervalo, fecha1, fecha2)	Diferencia entre fechas	`DATEDIFF(DAY, '2025-04-01', '2025-04-14') → 13`

2.9. SUBCONSULTAS Y CONSULTAS ANIDADAS

En SQL, una **subconsulta** (también llamada **consulta anidada**) es una instrucción `SELECT` que se **incluye dentro de otra consulta principal**. Sirve para **obtener resultados intermedios** que se utilizan como filtro, comparación o fuente de datos en la consulta externa.

Las subconsultas permiten escribir consultas complejas de forma **modular y más legible**, dividiendo el problema en partes. Pueden situarse en cláusulas como `WHERE`, `FROM`, `SELECT` o incluso en `HAVING`.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

```
SELECT columnas
FROM tabla
WHERE columna = (
  SELECT columna
  FROM otra_tabla
  WHERE condición
);
```

Ejemplo: Obtener los empleados que trabajan en el departamento llamado 'Marketing', En este ejemplo, la subconsulta obtiene el ID del departamento 'Marketing', y ese valor se utiliza en la consulta principal para filtrar empleados.

```
SELECT Nombre
FROM Empleados
WHERE DepartamentoID = (
  SELECT ID
  FROM Departamentos
  WHERE Nombre = 'Marketing'
);
```

Tipos de subconsultas

1. Subconsulta escalar. Devuelve un único valor (una fila, una columna). Se usa en comparaciones simples o asignaciones.

Ejemplo: Devuelve los empleados que cobran más que la media.

```
SELECT Nombre
FROM Empleados
WHERE Salario > (
  SELECT AVG(Salario)
  FROM Empleados
);
```


© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2. Subconsulta de conjunto (tabla). Devuelve varias filas y columnas.

Ejemplo con IN:

```
SELECT Nombre
FROM Empleados
WHERE DepartamentoID IN (
  SELECT ID
  FROM Departamentos
  WHERE Ubicacion = 'Madrid'
);
```

Ejemplo en FROM:

```
SELECT AVG(Salario)
FROM (
  SELECT Salario
  FROM Empleados
  WHERE DepartamentoID = 10
) AS Subconsulta;
```



Se usa habitualmente:

- IN (...)
- Comparaciones múltiples (= ANY, = ALL)
- En el FROM como si fuera una tabla derivada.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

3. Subconsulta correlacionada. Es una subconsulta que se ejecuta una vez por cada fila de la consulta externa, ya que depende de columnas de la consulta principal. Se comporta como un bucle interno**.**

Ejemplo: Por cada empleado, se calcula la media salarial de su departamento y se compara con su propio salario.

```
SELECT E.Nombre

FROM Empleados E

WHERE Salario > (

  SELECT AVG(Salario)

  FROM Empleados

  WHERE DepartamentoID = E.DepartamentoID

);
```

2.10. VISTAS, ÍNDICES Y RESTRICCIONES

SQL permite definir **objetos estructurales** que mejoran la **organización, el rendimiento y la integridad** de las bases de datos.

- **Vistas (VIEW)**

Una **vista** es una **consulta almacenada** que actúa como una **tabla virtual**. No guarda datos físicamente, sino que muestra resultados generados a partir de otras tablas.

```
CREATE VIEW nombre_vista AS

SELECT columnas

FROM tabla

WHERE condición;
```

Ejemplo: Esta vista muestra solo los empleados activos.

```
CREATE VIEW VistaActivos AS

SELECT * FROM Empleados

WHERE Activo = TRUE;
```

- **Índices (INDEX)**

Un **índice** es una estructura que mejora el rendimiento de las consultas al **acelerar la búsqueda** de valores en una o varias columnas. Funciona de forma similar a un índice de libro.

```
CREATE INDEX nombre_indice

ON tabla (columna);
```


© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Ejemplo: Crea un índice sobre la columna Nombre de la tabla Empleados.

```
CREATE INDEX idx_nombre
ON Empleados (Nombre);
```

Tipos de índices:

- **Únicos (UNIQUE)**: impiden valores duplicados.
- **Compuestos**: involucran varias columnas.
- **Clustered** y **Non-Clustered** (según el SGBD).

- **Restricciones (CONSTRAINTS)**

Las **restricciones** son reglas que se aplican a las columnas o tablas para **garantizar la integridad de los datos**. Se pueden definir al crear una tabla o añadir posteriormente.

Restricción	Descripción
PRIMARY KEY	Identifica unívocamente cada fila. No permite valores NULL.
FOREIGN KEY	Establece una relación con otra tabla. Mantiene coherencia entre datos.
UNIQUE	Garantiza que todos los valores de una columna sean distintos.
NOT NULL	Impide que una columna tenga valores nulos.
CHECK	Valida una condición lógica sobre los datos.

3. LENGUAJES DE BASES DE DATOS

3.1. LENGUAJE DE DEFINICIÓN DE DATOS (DDL)

El **DDL (Data Definition Language)** permite definir y modificar la estructura de la base de datos. Afecta al **esquema**, no a los datos directamente.

A. CREATE

La instrucción **CREATE** se utiliza para **crear nuevos objetos en la base de datos**. Es una de las sentencias fundamentales en SQL, ya que permite definir la **estructura inicial** que dará soporte al almacenamiento y gestión de los datos.

Con CREATE se pueden crear diferentes tipos de objetos:

- **Tablas (CREATE TABLE)**: estructuras que almacenan los datos en forma de filas y columnas.
- **Vistas (CREATE VIEW)**: representaciones virtuales de datos que resultan de una consulta.
- **Índices (CREATE INDEX)**: estructuras que mejoran el rendimiento de búsquedas.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Esquemas y usuarios**: en algunos SGBD también se pueden crear espacios lógicos y usuarios mediante CREATE.
- **Procedimientos y funciones almacenadas** (en combinación con extensiones del lenguaje): permiten encapsular lógica de negocio.



Para la creación de una **tabla**, se utiliza la siguiente sintaxis básica:

```
CREATE TABLE nombre_tabla (
  columna1 tipo_dato [restricciones],
  columna2 tipo_dato [restricciones],
  ...
);
```

- El **nombre de la tabla**.
- Los **nombres de las columnas**.
- El **tipo de datos** de cada columna.
- Opcionalmente, restricciones como claves primarias, foráneas, unicidad o valores por defecto.

Ejemplo: Crea una tabla llamada Empleados con una clave primaria, una columna obligatoria (Nombre) y otras opcionales. A partir de esta estructura, se podrán insertar y consultar datos.

```
CREATE TABLE Empleados (
  ID INT PRIMARY KEY,
  Nombre VARCHAR(100) NOT NULL,
  FechaIngreso DATE,
  DepartamentoID INT
);
```


© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

B. ALTER

La instrucción **ALTER** se utiliza para **modificar la estructura de objetos existentes** en la base de datos, especialmente de tablas. Es fundamental para adaptar el diseño de un esquema sin necesidad de eliminar y volver a crear objetos. Con ALTER se pueden realizar diversas operaciones:

- **Añadir nuevas columnas** a una tabla.
- **Modificar el tipo de dato o las restricciones** de una columna existente.
- **Eliminar columnas**.
- **Cambiar el nombre de columnas o tablas** (según el SGBD).
- **Agregar o eliminar restricciones**, como claves primarias, foráneas o UNIQUE.

Operación	Comando SQL	
Añadir nuevas columnas a una tabla	ADD	ALTER TABLE nombre_tabla ADD nombre_columna tipo_dato [restricciones];
Modificar el tipo de dato o las restricciones de una columna existente	MODIFY (o ALTER COLUMN)	ALTER TABLE nombre_tabla **MODIFY** nombre_columna nuevo_tipo_dato [nuevas_restricciones];
Eliminar columnas	DROP COLUMN	ALTER TABLE nombre_tabla **DROP COLUMN** nombre_columna;
Cambiar el nombre de una columna (según el SGBD, por ejemplo MySQL ≥ 8.0)	RENAME COLUMN	ALTER TABLE nombre_tabla **RENAME COLUMN** nombre_antiguo TO nombre_nuevo;
Cambiar el nombre de una tabla	RENAME TO	ALTER TABLE nombre_antiguo **RENAME TO** nombre_nuevo;

El uso de ALTER permite que el diseño de la base de datos evolucione conforme cambian los requisitos del sistema, sin afectar directamente a los datos ya almacenados (salvo que el cambio implique incompatibilidades).

Para modificar una **tabla**, se utilizan algunas de las siguientes instrucciones:

```
ALTER TABLE nombre_tabla
  ADD nombre_columna tipo_dato [restricciones];
ALTER TABLE nombre_tabla
  MODIFY nombre_columna tipo_dato;
ALTER TABLE nombre_tabla
  DROP COLUMN nombre_columna;
```

Ejemplos:

- ***Añadir una columna:***

```
ALTER TABLE Empleados
ADD Telefono VARCHAR(20);
```

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- ***Modificar el tipo de dato de una columna:***

```
ALTER TABLE Empleados

MODIFY Telefono VARCHAR(30);
```

- ***Eliminar una columna:***

```
ALTER TABLE Empleados

DROP COLUMN Telefono;
```

C. DROP

La instrucción **DROP** se utiliza para **eliminar completamente un objeto** de la base de datos. Este objeto puede ser una tabla, una vista, un índice, un procedimiento, una función, un esquema, etc.

Cuando se ejecuta un DROP, el objeto desaparece **de forma permanente**, lo que implica la **pérdida total de su estructura y de todos los datos que pudiera contener** (en el caso de una tabla). No puede recuperarse salvo que exista una copia de seguridad.

```
DROP TABLE nombre_tabla;

DROP VIEW nombre_vista;

DROP INDEX nombre_indice;
```



En algunos SGBD se puede usar la cláusula IF EXISTS para evitar errores si el objeto no existe:

DROP TABLE IF EXISTS Empleados;

Si existen restricciones referenciales (claves foráneas en otras tablas), el SGBD puede impedir el borrado salvo que estas relaciones se eliminen antes o se use CASCADE.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

D. TRUNCATE

La instrucción **TRUNCATE** se utiliza para **eliminar todos los registros** de una tabla, sin borrar su definición. Es decir, **vacía la tabla por completo**, pero **mantiene su estructura**, lo que permite reutilizarla inmediatamente.

Se diferencia de DELETE en que:

- TRUNCATE **no permite usar cláusulas WHERE**: elimina todo.
- Es **más rápida y eficiente**, ya que opera a nivel de páginas de datos.
- En la mayoría de SGBD, **no activa triggers** ni registra fila por fila en el log de transacciones.

```
TRUNCATE TABLE nombre_tabla;
```

3.2. LENGUAJE DE MANIPULACIÓN DE DATOS (DML)

El **DML (Data Manipulation Language)** es el subconjunto de SQL que permite **gestionar los datos contenidos en las tablas**, sin alterar su estructura. A través de estas instrucciones, el usuario puede insertar nuevos registros, actualizar los existentes o eliminar aquellos que ya no se necesitan.

A. INSERT



La instrucción **INSERT** se utiliza para **añadir nuevas filas (registros)** a una tabla existente. Puede especificarse el valor de todas o algunas columnas.

```
INSERT INTO nombre_tabla (columna1, columna2, ...)
VALUES (valor1, valor2, ...);
```

Si no se indica el nombre de las columnas, se insertan los campos en el orden especificado.

```
INSERT INTO nombre_tabla
VALUES (valor1, valor2, ...);
```

La combinación de las instrucciones INSERT y SELECT permite **insertar en una tabla los resultados obtenidos de una consulta** sobre otra u otras tablas.

A diferencia de INSERT … VALUES, que añade una fila con datos explícitos, esta combinación de instrucciones **toma las filas resultantes de una consulta** y las introduce directamente en la tabla destino. Es necesario que la **cantidad y el tipo de columnas** de la tabla destino coincidan con las columnas que devuelve el SELECT.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

```
INSERT INTO tabla_destino (columna1, columna2, ...)
SELECT columna1, columna2, ...
FROM tabla_origen
[WHERE condición];
```

B. UPDATE

La instrucción **UPDATE** permite **modificar los valores de una o varias columnas** en las filas que cumplan una determinada condición. Si no se especifica un WHERE, **todas las filas** de la tabla serán modificadas.

```
UPDATE nombre_tabla
SET columna1 = valor1 [, columna2 = valor2, ...]
WHERE condición;
```

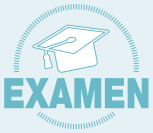
- **UPDATE nombre_tabla**. Este es el primer componente de la instrucción. Indica la tabla sobre la que se va a realizar la actualización. Solo puede actualizarse una tabla por instrucción UPDATE.
- **SET columna = nuevo_valor**. La cláusula SET especifica qué columnas van a modificarse y con qué nuevos valores. *Pueden modificarse varias columnas en la misma sentencia separándolas por comas.*

  SET Nombre = 'María', DepartamentoID = 20

  El valor asignado puede ser:

  - Un valor constante.
  - Una expresión o función.
  - El resultado de una subconsulta (en algunos casos).
- **WHERE condición (opcional)**. La cláusula tebob indica qué filas deben actualizarse. Si se omite, todas las filas de la tabla serán modificadas, lo cual suele ser peligroso si no se desea un cambio masivo.

C. DELETE



La instrucción **DELETE** elimina filas de una tabla que cumplan una condición especificada. Es más flexible que TRUNCATE, ya que permite eliminar registros específicos mediante WHERE.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

```
DELETE FROM nombre_tabla

WHERE condición;
```

- Sin WHERE, se eliminarán **todas las filas** de la tabla (como con TRUNCATE, pero más lento y registrando cada eliminación).
- Puede activar **triggers** definidos sobre DELETE.
- La operación puede deshacerse si está dentro de una transacción.

3.3 LENGUAJE DE CONTROL DE DATOS (DCL)

El **Lenguaje de Control de Datos (DCL)** permite **gestionar la seguridad** dentro de una base de datos, mediante el **otorgamiento o revocación de permisos** sobre los objetos definidos (tablas, vistas, procedimientos, etc.). Es esencial para establecer políticas de acceso por usuario o por rol.

A. GRANT

La instrucción **GRANT** permite **conceder permisos** a uno o varios usuarios sobre objetos específicos de la base de datos. Estos permisos controlan qué operaciones puede realizar un usuario: lectura, escritura, ejecución, etc.

```
GRANT privilegios

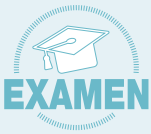
ON objeto

TO usuario [, ...];
```

Privilegios comunes:

- SELECT: permite consultar datos.
- INSERT: permite insertar datos.
- UPDATE: permite modificar datos.
- DELETE: permite eliminar datos.
- ALL: concede todos los privilegios anteriores.

B. REVOKE



La instrucción **REVOKE** sirve para **retirar permisos** previamente otorgados a un usuario o rol sobre un objeto de la base de datos.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

```
REVOKE privilegios
ON objeto
FROM usuario [, ...];
```

3.4. OQL: OBJECT QUERY LANGUAGE

OQL (Object Query Language) es un lenguaje de consultas diseñado para trabajar con **bases de datos orientadas a objetos**. Forma parte del estándar **ODMG (Object Data Management Group)** y está inspirado en el estilo declarativo de SQL, pero adaptado a las **estructuras y características del paradigma orientado a objetos**, como clases, objetos, herencia y métodos.

- Basado en SQL pero orientado a objetos.
- Permite consultar **instancias de clases**, incluyendo atributos complejos y relaciones entre objetos.
- Admite **tipos compuestos** (colecciones, arreglos, conjuntos, listas).
- Utiliza **cláusulas similares a SQL**: SELECT, FROM, WHERE, GROUP BY, ORDER BY.
- Soporta **navegación entre objetos** mediante referencias o caminos (obj.subobj. propiedad).

La estructura general de una consulta OQL es similar a SQL:

```
SELECT expresión
FROM nombre_clase [AS] alias
WHERE condición
```

Ejemplo: Supongamos que tenemos una base de datos orientada a objetos con una clase Empleado que tiene un atributo departamento de tipo Departamento. Esta consulta devuelve los nombres de los empleados cuyo departamento se llama "Marketing".

```
SELECT e.nombre
FROM Empleado e
WHERE e.departamento.nombre = "Marketing"
```

Ejemplo con colecciones: Si Empleado tiene un atributo proyectos que es una colección de objetos Proyecto. Aquí se seleccionan empleados que participan en el proyecto llamado "Proyecto A".

```
SELECT e.nombre
FROM Empleado e
WHERE "Proyecto A" IN e.proyectos.nombre
```

Supera. by thePower © Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

OQL fue pensado para:

- Consultar objetos en **bases de datos orientadas a objetos** como ObjectStore o Versant.
- Integrarse con lenguajes de programación orientados a objetos (C++, Java).
- Ser parte del estándar ODMG (hoy en desuso, pero influyente).

Actualmente, su uso es **residual** frente a lenguajes modernos como **JPQL** (Java Persistence Query Language), **HQL** (Hibernate Query Language) o expresiones LINQ, que recogen muchas de sus ideas en entornos relacionales u ORMs.

4. PROCEDIMIENTOS ALMACENADOS

Los **procedimientos almacenados** (o *stored procedures*) son bloques de código SQL que se almacenan y ejecutan en el propio servidor de base de datos. Permiten encapsular lógica de negocio, automatizar tareas y reutilizar operaciones complejas, ofreciendo ventajas tanto de rendimiento como de organización del código.

Un **procedimiento almacenado** es una **función definida por el usuario** que agrupa un conjunto de instrucciones SQL para ser ejecutadas de forma repetitiva o bajo demanda.

4.1. SINTAXIS BÁSICA (CREATE PROCEDURE)

La forma de definir un procedimiento varía según el SGBD, pero la estructura básica incluye:

- Nombre del procedimiento.
- Lista de parámetros (opcionales).
- Bloque de instrucciones SQL.

MySQL:

```
DELIMITER //

CREATE PROCEDURE NombreProcedimiento()

BEGIN

  -- Instrucciones SQL

END //

DELIMITER ;
```

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

SQL Server:

```
CREATE PROCEDURE NombreProcedimiento
AS
BEGIN
  -- Instrucciones SQL
END;
```

4.2. PARÁMETROS DE ENTRADA Y SALIDA

Los **procedimientos almacenados** pueden recibir y devolver datos mediante el uso de **parámetros**, lo que los hace **más flexibles y reutilizables**. Estos parámetros permiten que el procedimiento actúe de forma dinámica dependiendo de los valores que se le pasen o devuelvan.

Tipos de parámetros

Existen tres tipos de parámetros, definidos explícitamente al declarar el procedimiento.

Tipo	Descripción
IN	**Parámetro de entrada**: el procedimiento recibe un valor desde el exterior.
OUT	**Parámetro de salida:** el procedimiento devuelve un valor al exterior.
INOUT	**Parámetro mixto:** actúa como entrada y salida a la vez.

- **IN – Parámetros de entrada**

Permiten **pasar un valor al procedimiento**, que podrá utilizarse dentro de su cuerpo, pero no se puede modificar fuera de él.

MySQL:

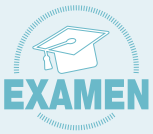
```
CREATE PROCEDURE MostrarEmpleado(IN empID INT)
BEGIN
  SELECT * FROM Empleados WHERE ID = empID;
END;
```

SQL Server:

```
CREATE PROCEDURE MostrarEmpleado
  @empID INT
AS
BEGIN
  SELECT * FROM Empleados WHERE ID = @empID;
END;
```


© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **OUT – Parámetros de salida**



Se utilizan **para devolver información desde el procedimiento** al llamante. Son útiles para cálculos, totales o consultas individuales.

Ejemplo en MySQL: Este procedimiento devuelve el salario del empleado indicado y lo almacena en el parámetro salario.

```
CREATE PROCEDURE ObtenerSalario(
  IN empID INT,
  OUT salario DECIMAL(10,2)
)
BEGIN
  SELECT Salario INTO salario FROM Empleados WHERE ID = empID;
END;
```

Ejemplo equivalente en SQL Server: (Al llamar al procedimiento, es necesario declarar una variable para recibir el valor de salida).

```
CREATE PROCEDURE ObtenerSalario
  @empID INT,
  @salario DECIMAL(10,2) OUTPUT
AS
BEGIN
  SELECT @salario = Salario FROM Empleados WHERE ID = @empID;
END;
```

- **INOUT – Parámetros mixtos (MySQL)**

Este tipo de parámetro permite **modificar un valor y devolverlo al llamante**, actuando como **entrada y salida simultáneamente**.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Ejemplo en MySQL: Este procedimiento incrementa el salario pasado como parámetro en un 10 % y devuelve el nuevo valor a la misma variable.

```
CREATE PROCEDURE AjustarSalario(INOUT salario DECIMAL(10,2))
BEGIN
  SET salario = salario * 1.10;
END;
```

- **Llamada a procedimientos con parámetros**

En MySQL:

```
CALL ObtenerSalario(3, @salarioResultado);
SELECT @salarioResultado;
```

En SQL Server:

```
DECLARE @resultado DECIMAL(10,2);
EXEC ObtenerSalario @empID = 3, @salario = @resultado OUTPUT;
SELECT @resultado;
```

4.3. VARIABLES LOCALES, CONDICIONALES Y BUCLES

Los procedimientos almacenados no solo ejecutan consultas, sino que también permiten implementar **lógica de programación estructurada**, utilizando **variables locales**, **estructuras condicionales** y **bucles**. Esto los convierte en potentes herramientas para automatizar procesos complejos directamente en la base de datos.

4.3.1. Variables locales

Las **variables locales** permiten almacenar y manipular datos dentro del procedimiento, como contadores, acumuladores, resultados de consultas, etc.

√ **MySQL:**

```
DECLARE nombre_variable tipo;
```

√ **SQL Server:**

```
DECLARE @nombre_variable tipo;
```


© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

4.3.2. Estructuras condicionales

- **IF ->** Permite ejecutar distintas instrucciones según se cumpla o no una condición.

√ **(MySQL)**

```
IF ... THEN ... ELSE
```

√ **(SQL Server)**

```
IF ... ELSE
```

Ejemplo en MySQL:

```
CREATE PROCEDURE CalcularBonus(IN empID INT)
BEGIN
  DECLARE salario_base DECIMAL(10,2);
  SELECT Salario INTO salario_base FROM Empleados WHERE ID = empID;
  IF salario_base < 30000 THEN
    UPDATE Empleados SET Bonus = 1000 WHERE ID = empID;
  ELSE
    UPDATE Empleados SET Bonus = 500 WHERE ID = empID;
  END IF;
END;
```

Ejemplo en SQL Server:

```
CREATE PROCEDURE CalcularBonus
  @empID INT
AS
BEGIN
  DECLARE @salario_base DECIMAL(10,2);
  SELECT @salario_base = Salario FROM Empleados WHERE ID = @empID;
  IF @salario_base < 30000
    UPDATE Empleados SET Bonus = 1000 WHERE ID = @empID;
  ELSE
    UPDATE Empleados SET Bonus = 500 WHERE ID = @empID;
END;
```

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **CASE – Evaluación de múltiples condiciones ->** Es útil cuando se desea comprobar varios casos posibles.

```
CASE ... WHEN . . . THEN ... ELSE
```

Ejemplo en MySQL:

```
SET salario_base = CASE
  WHEN salario_base < 20000 THEN 1500
  WHEN salario_base < 30000 THEN 1000
  ELSE 500
END;
```

Ejemplo en SQL Server:

```
SET @bonus = CASE
  WHEN @salario_base < 20000 THEN 1500
  WHEN @salario_base < 30000 THEN 1000
  ELSE 500
END;
```

4.3.3. Bucles

Permiten repetir un bloque de instrucciones mientras se cumpla una condición.

- **WHILE (disponible tanto en MySQL como en SQL Server)**

```
WHILE condición DO
   -- Instrucciones a repetir
END WHILE;
```

Ejemplo en MySQL:

```
CREATE PROCEDURE Contador()
BEGIN
  DECLARE i INT DEFAULT 1;
  WHILE i <= 5 DO
    INSERT INTO Log (Mensaje) VALUES (CONCAT('Iteración ', i));
    SET i = i + 1;
  END WHILE;
END;
```


© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Ejemplo en SQL Server:

```
CREATE PROCEDURE Contador
AS
BEGIN
  DECLARE @i INT = 1;
  WHILE @i <= 5
  BEGIN
    INSERT INTO Log (Mensaje) VALUES (CONCAT('Iteración ', @i));
    SET @i = @i + 1;
  END
END;
```

- **LOOP y REPEAT (solo en MySQL)**
  - **LOOP**: bucle básico.

```
nombre_bucle: LOOP
   -- Instrucciones
   IF condición_de_salida THEN
      LEAVE nombre_bucle;
   END IF;
END LOOP;
```

  - **REPEAT**: se ejecuta **al menos una vez**.

```
REPEAT
   -- Instrucciones
UNTIL condición_de_salida
END REPEAT;
```

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

5. EVENTOS Y DISPARADORES (TRIGGERS)

Los **eventos** y **disparadores** permiten que una base de datos **reaccione automáticamente ante ciertas acciones**, como la inserción, actualización o eliminación de datos. Son mecanismos clave para **automatizar procesos**, **validar reglas de negocio** y **garantizar la integridad** más allá de las restricciones estándar.

5.1. DEFINICIÓN DE EVENTOS Y TRIGGERS

A diferencia de los procedimientos almacenados, **los triggers no se llaman manualmente**, sino que se activan **de forma implícita** como respuesta a una acción.

- Un **disparador (trigger)** es un bloque de código SQL que se ejecuta **automáticamente cuando se produce un evento** (como un INSERT, UPDATE o DELETE) sobre una tabla.
- Un **evento**, en algunos SGBD como MySQL, también puede referirse a tareas programadas que se ejecutan en determinados momentos del tiempo, aunque aquí nos centraremos en triggers ligados a acciones sobre datos.

5.2. SINTAXIS DE UN TRIGGER EN SQL

La **sintaxis general** de un trigger (siguiendo el estilo ANSI SQL y compatible con sistemas como Oracle, MySQL o PostgreSQL con variaciones mínimas) es:

```
CREATE TRIGGER nombre_trigger

{BEFORE | AFTER} {INSERT | UPDATE [OF columna] | DELETE}

ON nombre_tabla

[FOR EACH ROW]

[WHEN (condición)]

BEGIN

  -- Instrucciones SQL a ejecutar

END;
```


© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Elemento	Descripción
CREATE TRIGGER	Palabras clave para definir un nuevo trigger.
nombre_trigger	Nombre identificativo del trigger. Debe ser único dentro del esquema.
[OF columna] (opcional)	En caso de UPDATE, se puede limitar el trigger a una **columna específica**.
ON nombre_tabla	Tabla (o vista) sobre la que se define el trigger.
FOR EACH ROW	El trigger se ejecuta **una vez por cada fila afectada** (si se omite, puede implicar ejecución por sentencia).
WHEN (condición) (opcional)	Permite ejecutar el trigger **solo si se cumple** una condición lógica.
BEGIN ... END	Delimita el **bloque de instrucciones SQL** a ejecutar automáticamente.

Acceso a valores antes y después del cambio

En triggers de tipo UPDATE y DELETE, se puede acceder a los **valores antiguos (OLD)** y **nuevos (NEW)** de cada columna:

Contexto	OLD.columna	NEW.columna
INSERT	❌ (no existe)	☑ disponible
UPDATE	☑ disponible	☑ disponible
DELETE	☑ disponible	❌ (no existe)

5.3. DISPARADORES BEFORE Y AFTER

En SQL, un **trigger** puede definirse para ejecutarse en **dos momentos distintos** respecto a la operación DML (Data Manipulation Language) que lo activa:

Tipo de trigger	Momento de ejecución	Uso principal
BEFORE	Antes de realizar la operación	Validar, modificar datos antes de insertarlos
AFTER	Después de realizar la operación	Registrar lo que ha ocurrido durante la transacción

5.4. PSEUDOREGISTROS NEW Y OLD

Cuando se produce un evento (INSERT, UPDATE o DELETE), SQL proporciona accesos especiales a los valores implicados en la fila mediante dos pseudoregistros:

Pseudoregistro	Cuándo se usa	Contiene...
NEW.columna	En INSERT, UPDATE	El valor nuevo asignado a la columna
OLD.columna	En UPDATE, DELETE	El valor antiguo antes de la modificación

Ejemplo en MySQL:

- ***Se ejecuta automáticamente después de actualizar un registro en la tabla Empleados.***
- ***Para cada fila afectada (FOR EACH ROW), inserta una entrada en la tabla LogCambios.***
- ***Registra:***
  - ***El ID del empleado (OLD.ID).***

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- *La fecha y hora exactas del cambio (NOW()).*
- *El valor del salario antes de la actualización (OLD.Salario).*
- *El nuevo valor del salario (NEW.Salario).*

Este tipo de trigger es útil para mantener un historial de cambios sin tener que modificar manualmente la tabla LogCambios cada vez que se hace un UPDATE.

```
CREATE TRIGGER trg_auditoria

AFTER UPDATE ON Empleados

FOR EACH ROW

INSERT INTO LogCambios (EmpleadoID, FechaCambio, SalarioAnterior,
SalarioNuevo)

VALUES (OLD.ID, NOW(), OLD.Salario, NEW.Salario);
```

5.5. TIPOS DE EVENTOS: INSERT, UPDATE, DELETE

Los triggers se pueden asociar a los siguientes **eventos DML**:

Evento	Descripción
INSERT	Se activa al **añadir nuevos registros.**
UPDATE	Se activa al **modificar registros existentes.**
DELETE	Se activa al **eliminar registros.**

Cada trigger puede utilizar las palabras clave:

- NEW.columna → valor nuevo (en INSERT o UPDATE)
- OLD.columna → valor antiguo (en UPDATE o DELETE)

Los triggers se utilizan habitualmente en los siguientes contextos:

- **Auditoría: Registrar cambios en los datos**

```
AFTER UPDATE → INSERT INTO LogCambios(…)
```

- **Validación: Evitar inserciones o modificaciones erróneas:**

```
BEFORE INSERT → IF NEW.Valor < 0 THEN SIGNAL ERROR
```

- **Automatización: Actualizar campos derivados o sincronizar tablas:**

```
AFTER INSERT → UPDATE TablaSecundaria SET ...

...
```


© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

6. CONTROL DE TRANSACCIONES Y NIVELES DE AISLAMIENTO

Una **transacción** es una unidad lógica de trabajo que agrupa una o varias instrucciones SQL, las cuales deben ejecutarse **completamente o no ejecutarse en absoluto**, garantizando así la **integridad de los datos**.

Las transacciones siguen las propiedades **ACID**: Atomicidad, Consistencia, Aislamiento e Durabilidad.

6.1. INSTRUCCIONES DE CONTROL DE TRANSACCIONES

El estándar ANSI SQL define las siguientes sentencias básicas para el manejo de transacciones:

```
BEGIN TRANSACTION;    -- Inicia una transacción

-- Instrucciones SQL

COMMIT;               -- Confirma los cambios

ROLLBACK;             -- Deshace los cambios si hubo errores
```

Estas instrucciones pueden usarse también dentro de procedimientos almacenados.

6.2. PROBLEMAS DE CONCURRENCIA



Cuando varias transacciones se ejecutan simultáneamente sobre los mismos datos, pueden surgir anomalías si no se controla adecuadamente el acceso concurrente:

Fenómeno	Descripción
Lectura sucia	Una transacción lee datos que otra aún no ha confirmado.
Lectura no repetible	Una transacción vuelve a leer un valor que ha sido modificado por otra.
Lectura fantasma	Una transacción vuelve a ejecutar una consulta y encuentra nuevas filas.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

6.3. NIVELES DE AISLAMIENTO DE TRANSACCIONES (ANSI SQL)

El estándar define cuatro niveles de aislamiento que determinan qué anomalías se permiten o se evitan durante la concurrencia:

Nivel de aislamiento	Lectura sucia	Lectura no repetible	Lectura fantasma
READ UNCOMMITTED	☑ permitida	☑ permitida	☑ permitida
READ COMMITTED	✗ evitada	☑ permitida	☑ permitida
REPEATABLE READ	✗ evitada	✗ evitada	☑ permitida
SERIALIZABLE	✗ evitada	✗ evitada	✗ evitada

6.4. CONTROL DE TRANSACCIONES: SENTENCIAS CLAVE EN SQL

Las **transacciones** permiten ejecutar un conjunto de operaciones como una unidad atómica. Para gestionarlas correctamente, SQL ofrece un conjunto de sentencias que controlan su inicio, confirmación, cancelación o recuperación parcial.

- **Sentencias básicas de control de transacciones**

Sentencia	Descripción
BEGIN / START TRANSACTION	Inicia una transacción. Todas las operaciones siguientes se agrupan hasta el COMMIT o ROLLBACK.
COMMIT	Confirma y guarda permanentemente todos los cambios realizados durante la transacción.
ROLLBACK	Cancela la transacción y revierte todos los cambios realizados desde el último BEGIN.

- **SAVEPOINT: crear un punto de recuperación**

  La sentencia SAVEPOINT permite establecer un **punto intermedio dentro de una transacción**, al que se puede volver en caso de error. No revierte toda la transacción, sino solo hasta el punto guardado.

```
SAVEPOINT nombre_punto;
```

  - **ROLLBACK TO: revertir hasta un punto específico**

  Permite deshacer parcialmente una transacción hasta un SAVEPOINT determinado, manteniendo los cambios anteriores.

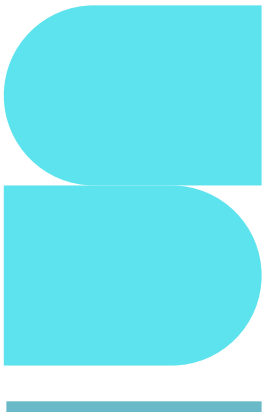
```
ROLLBACK TO nombre_punto;
```

  - **RELEASE SAVEPOINT (opcional)**

  En algunos SGBD (como PostgreSQL o Oracle), se puede liberar un SAVEPOINT si ya no se necesita.

```
RELEASE SAVEPOINT nombre_punto;
```


© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

Queda rigurosamente prohibida cualquier forma de reproducción, distribución comunicación pública o transformación total o parcial de esta obra sin el permiso escrito de los titulares de los derechos de explotación.